

Синтез и анализ на алгоритми за факултети ФКСТ, ФТК, ФЕТТ

проф. д-р Даниела Минковска
каб. 2324, кат. ПКТ, ФКСТ, ТУ-София
daniela@tu-sofia.bg



Лекция 3

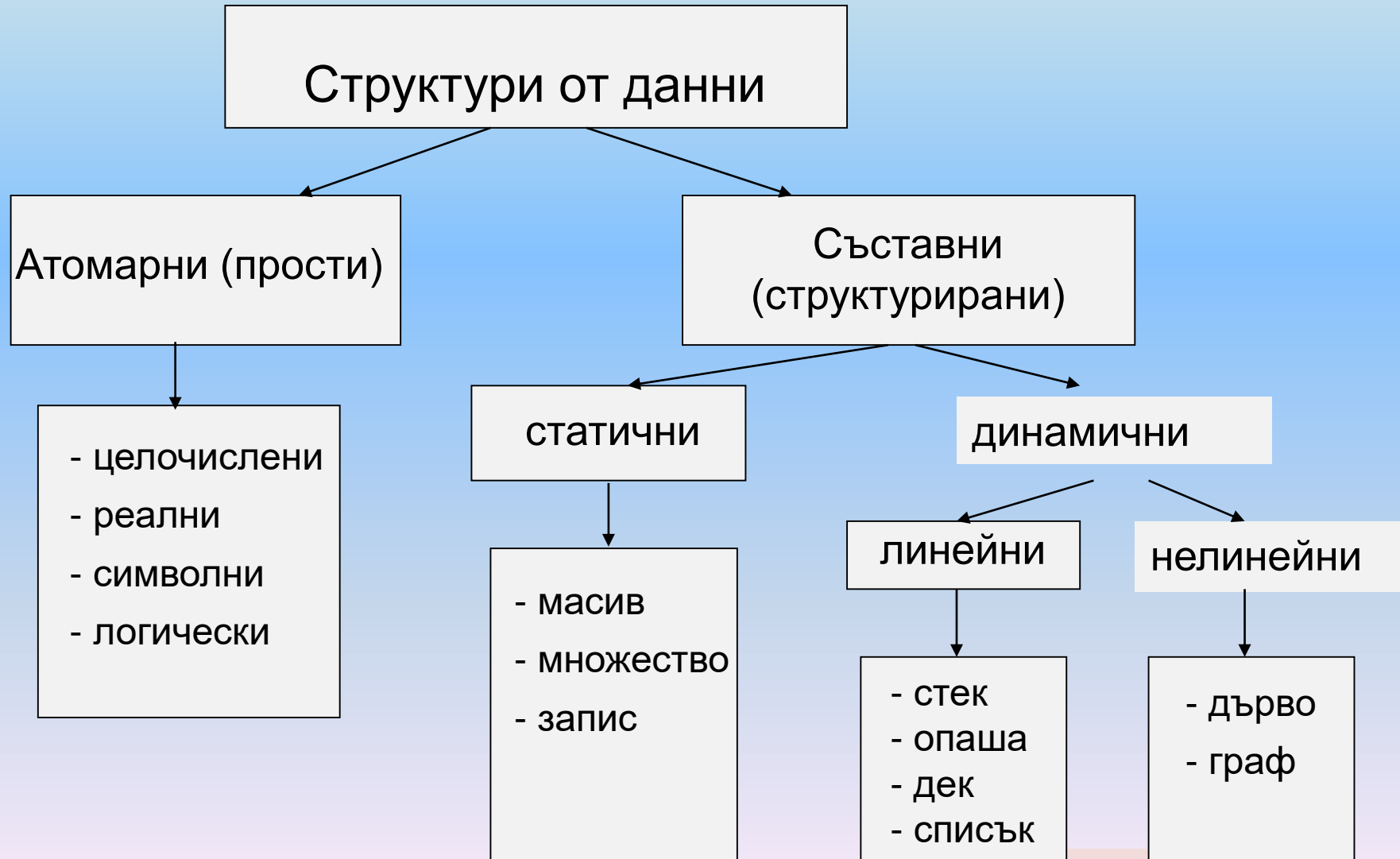
Структури от данни. Стек, опашка, списък – алгоритми за запис и четене на елемент.

“АЛГОРИТМИ + СТРУКТУРИ ОТ ДАННИ = ПРОГРАМИ”

Н. УИРТ



Структури от данни. Класификация.





Структури от данни.

Структурата от данни определя типа на данните и взаимното им разположение, т.е. **достъпа до данните**.

Най-често използваните структури са масив, стек, опашка, списък, дървета и графи. Използват се и вложени структури, при които елемент на дадена структура е от тип друга структура (*напр. масив от списъци*)

Изборът на подходяща структура от данни за даден алгоритъм има важно значение за неговата ефективност - бързодействие и заемна памет!



Структури от данни. Масиви

- **Дефиниция** - статичен, структуриран тип данни, представляващ **подредена** структура от **краен** брой **еднотипни** данни, наречени **елементи** на масива. Елементите на масива могат да бъдат от всеки допустим тип – **скаларен, структуриран или указател**.
- **Размер на масива:** Размерът на масива определя **максималния** брой данни, които масивът може да съдържа. Размерът на масива е **предварително фиксиран**.
- **Размерност на масива:** Размерността на масива е равна на **броя на индексите**, определящи (описващи) неговата структура.
- **Видове масиви** – **едномерни** и **двумерни**



Структури от данни. Едномерни масиви

- **Дефиниция** – Масив, който при описанието се задава с един индекс се нарича едномерен масив.

R → 3.6 8.4 2.4 1.2 4.8 5.4

R[I] → R[0] R[1] R[2] R[3] R[4] R[5]

Индексите на един масив с **n** елемента започват от **0** и завършват с **n-1**



Структури от данни. Двумерен масив

- **Дефиниция** – Масив, който при описанието се задава **с два индекса** се нарича двумерен масив.

A[4] , [5] →	66	12	78	15	99
	71	23	45	92	44
	24	88	64	12	31
	36	32	11	77	31

A[I] , [J] →	A[0][0]	A[0][1]	A[0][2]	A[0][3]	A[0][4]
	A[1][0]	A[1][1]	A[1][2]	A[1][3]	A[1][4]
	A[2][0]	A[2][1]	A[2][2]	A[2][3]	A[2][4]
	A[3][0]	A[3][1]	A[3][2]	A[3][3]	A[3][4]

Структури от данни. Деклариране на масив

- Синтаксис:

<тип> <име на масив> [брой елементи] ;

Забележки:

1. **<тип>** – всички типове на C;
2. **<име на масив>** – съставя се по правилата за съставяне на идентификатори;
3. **[брой елементи]** – израз от цял тип, който може да има само положителни стойности и не може да бъде променлива, определя броя на елементите на масива.

Достъпът до елемент на масива се осъществява посредством индекс (или адрес), като всички елементи на масива имат едно и също име (това на масива) но различни индекси.



Структури от данни. Примери за декларация на масиви

int a[10];

//едномерен масив с 10 цели числа

int a[10][10];

//двумерен масив с цели числа. 10 реда и 10 стъба.

float b[12];

//едномерен масив с 12 реални числа

char c[60];

//едномерен масив със символи.

Последният елемент се използва за служебни цели в него се зарежда символната константа '\0' за край на низ.

double any[365]

//едномерен масив с 365 реални числа



Структури от данни. Инициализиране на масив

тип идентификатор[размер₁]...[размер_n] = {списък от стойности};

int a[5] = {1, 2, 3, 4, 5};

float b[2][3] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0}; **/* По редове */**

или

float b[2][3] = {{1.0, 2.0, 3.0}, {4.0, 5.0, 6.0}};



Структури от данни. Обработка на едномерни масиви

- **Въвеждане на стойности на масив с 10 цели елементи :**

```
printf ("Vavedete elementite na masiva A \n");  
for (i=0;i<10;i++)  
{  
    printf ("Vavdete a[ %d ] chislo:", i);  
    scanf ("%d", &a[i]);  
}
```

- **Извеждане на стойностите на масив с 10 цели елементи:**

```
printf ("Izvejdane na vhodnite elementi na masiva A \n");  
for (i=0;i<10;i++)  
    printf ("a[%d]= %d \n",i, a[i]);
```



Структури от данни. Обработка на двумерни масиви

- **Въвеждане на стойности на двумерен масив с 2 реда и 3 стълба:**

```
printf ("Vavedete elementite na masiva A \n");  
for (i=0;i<2;i++)  
    for (j=0;j<3;j++)  
    {    printf ("Vavdete a[%d],[%d] chislo:", i,j);  
        scanf ("%d", &a[i][j]);  
    }
```

- **Извеждане на стойностите на двумерен масив с 2 реда и 3 стълба:**

```
printf ("Izvejdane na vhodnite elementi na masiva A \n");  
for (i=0;i<2;i++)  
    {    for (j=0;j<3;j++)  
        printf ("%d ",a[i][j]);  
        printf ("\n");  
    }
```



Структури от данни. Примери

Пример1: Реализация на класическия алгоритъм *сито на Ератостен* (III век преди н. е.) за намиране на всички прости числа в интервал от 0 до n :

	2	3	4	5	6	7	8	9	10	Prime numbers
11	12	13	14	15	16	17	18	19	20	
21	22	23	24	25	26	27	28	29	30	
31	32	33	34	35	36	37	38	39	40	
41	42	43	44	45	46	47	48	49	50	
51	52	53	54	55	56	57	58	59	60	
61	62	63	64	65	66	67	68	69	70	
71	72	73	74	75	76	77	78	79	80	
81	82	83	84	85	86	87	88	89	90	
91	92	93	94	95	96	97	98	99	100	
101	102	103	104	105	106	107	108	109	110	
111	112	113	114	115	116	117	118	119	120	



Структури от данни. Примери

Пример1: Реализация на класическия алгоритъм *сито на Ератостен* (III век преди н. е.) за намиране на всички прости числа в интервал от 0 до n :

```
#include <stdio.h>
#define n 100
main()
{
    int i, j, a[n];
    for (i=2; i<n; i++) a[i]=1;
    for (i=2; i<n; i++)
        if (a[i])
            for (j=i; i*j<n; j++) a[i*j]=0;
    printf ("%d ", 1);
    for (i=2; i<n; i++)
        if (a[i]) printf ("%d ", i);
    return 0;
}
```



Структури от данни. Примери

Пример2: Динамично заделяне на памет за масив:

```
#include <stdio.h>
main()
{
    int n, i, j;
    printf ("Input n:");
    scanf ("%d", &n);
    int *a=new int [n];
    if (a==NULL)
    {printf ("%s", "Недостиг на памет!");}
    else
    {
        for (i=2; i<n; i++)    *(a+i)=1;
        for (i=2; i<n; i++)
            if (*(a+i))
                for (j=i; i*j<n; j++)    *(a+i*j)=0;
        printf ("%d" "1 ");
        for (i=2; i<n; i++)
            if (*(a+i))    printf ("%d", i);
    }
    delete []a;
    return 0;
}
```



Структури от данни. Статични структури

Недостатъци на статичните структури от данни:

1. Почти всички структури имат фиксиран брой елементи;
2. Структурите трудно се обновяват (добавяне и премахване на елементи);
3. Всички статични структури използват фиксиран обем оперативна памет, която не винаги се използва ефективно.

Динамичните структури от данни **не се описват**, а се създават (и унищожават) по време на изпълнение на програмата.

Структури от данни. Стек, опашка, списък

Търсенето в различните структури се извършва по различни алгоритми.

Стекът, опашката, декът и списъкът са **линейни** структури – при тях, всеки елемент има най-много два съседни елемента (предишен и следващ); само първият и последният (*началният и крайният*) елементи имат по един съседен елемент.

Основни операции:

- **Запис** (вмъкване) на нов елемент в структурата;
- **Четене** на елемент (обикн. е съпроводено с **изтриване** на елемент от структурата);
- **Търсене** в структурата на елемент със зададена стойност.



Структури от данни. Стек, опашка, списък

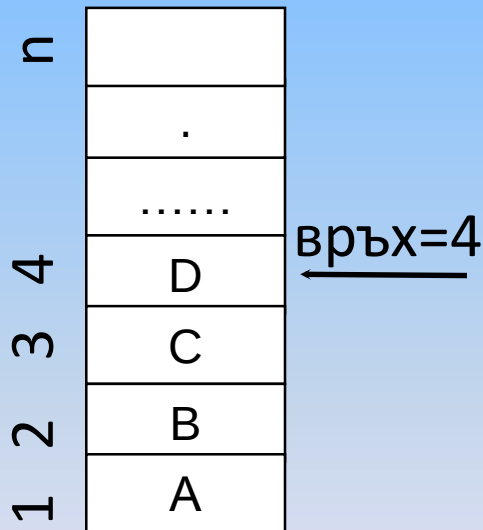
Основните операции могат да се представят по **два начина**:

- **Статично (с масив)** - заеманата от структурата памет по време на изпълнение на задачата не се променя;
- **Динамично (с указатели)** – паметта се променя в зависимост от големината на структурата – паметта нараства при нарастване на размера на структурата и респ., паметта намалява при намаляване на размера на структурата.



Структури от данни. Стек (Stack)

- **Стек** - линейна структура, подредена последователност от еднотипни елементи в която записът и четенето се извършват в единия край, наречен **връх**. Другият край на стека се нарича **дъно**. Стекът работи по принципа **LIFO** - Last In First Out.



Стекът е **празен** при **връх = 0**; и е **пълен** при **връх = n**

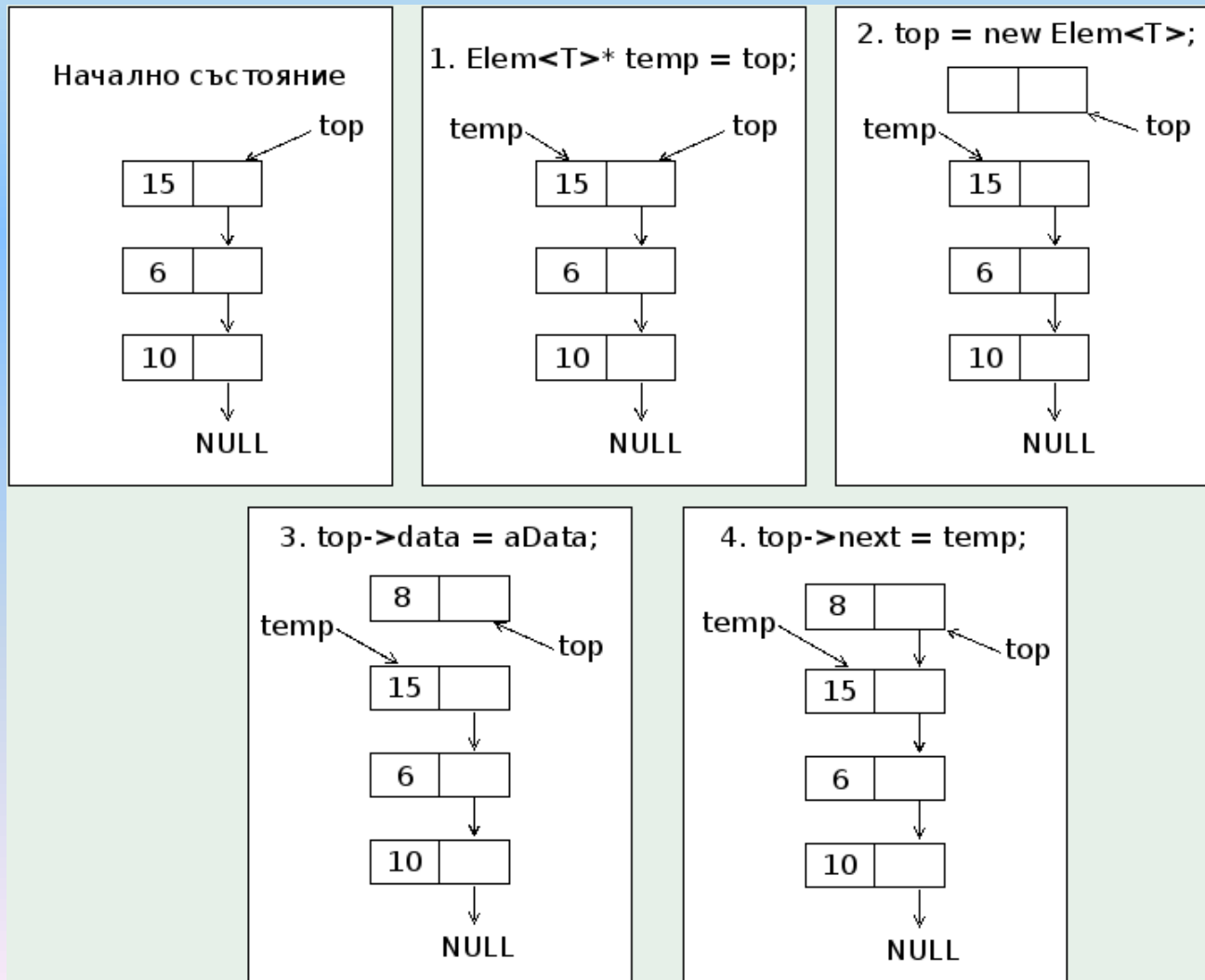
Основни операции със стек:

- Инициализиране на стека (**init**);
- Добавяне на елемент в стека (**push**);
- Извличане на елемент от стека (**pop**).



Структури от данни. Стек (Stack)

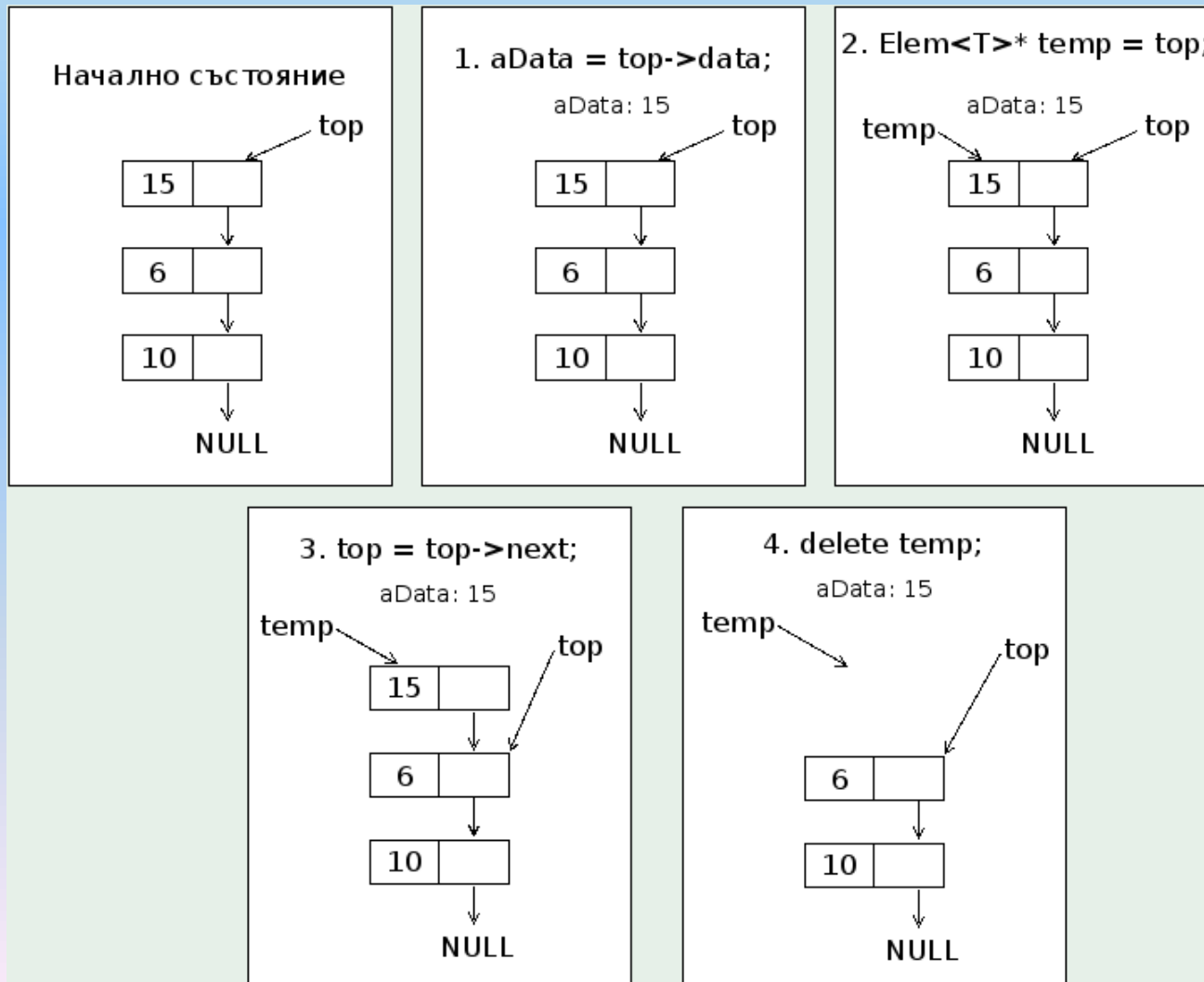
Основни стъпки при добавяне на елемент в стек (методът **push**):





Структури от данни. Стек (Stack)

Основни стъпки при извличане на елемент от стек (методът **pop**):





Структури от данни. Стек (Stack)

Пример: демонстрация на използване на стека (Във файла [stack.h](#) е реализиран класът *Stack*):

```
#include <iostream>
```

```
#include "stack.h"
```

```
using namespace std;
```

```
template <class T>
```

```
Stack<T> reverse(Stack<T> a) {
```

```
    T x;
```

```
    Stack<T> b;
```

```
    while(a.pop(x)) b.push(x);
```

```
    return b; }
```

```
int main() {
```

```
    Stack<int> s1;
```

// тук се вика конструкторът по подразбиране

```
    s1.push(2); cout << "push 2\n";
```

```
    s1.push(4); cout << "push 4\n";
```

```
    s1.push(6); cout << "push 6\n";
```

```
    cout << "s1:";
```

```
    s1.print();
```

```
    cout << endl;
```





Структури от данни. Стек (Stack)

```
cout << "Stack<int> s2(s1), s3;\ns3 = s1;\ns3.push(7);\n";  
Stack<int> s2(s1), s3; // тук се вика копиращият конструктор  
s3 = s1; // тук се вика операторът за присвояване  
s3.push(7);  
cout << "s1:" << s1 << endl; // тук се вика операторът за извеждане  
cout << "s2:" << s2 << endl;  
cout << "s3:" << s3 << endl;  
int a;  
s1.pop(a);  
cout << "\ns1.pop(a), a: " << a;  
cout << "\ns3.writeFile\n";  
s3.writeToFile("test.mp3");  
cout << "s1.readFile\n";  
s1.readFile("test.mp3");  
cout << "s1:" << s1 << endl;  
cout << "s2 = s1 = s3\n"; s2 = s1 = s3;  
cout << "\ns1:" << s1;  
cout << "\ns2:" << s2;  
cout << "\ns3:" << s3;
```





Структури от данни. Стек (Stack)

```
cout << "\ns2.push(15)\n"; s2.push(15);
```

```
cout << "\ns1:" << s1;
```

```
cout << "\ns2:" << s2;
```

```
cout << "\ns3:" << s3;
```

```
cout << "\ns1.pop(): " << s1.pop(); cout << "\ns1:" << s1 << "\n";
```

```
cout << "\ns1.pop(): " << s1.pop(); cout << "\ns1:" << s1 << "\n";
```

```
cout << "\ns1.pop(): " << s1.pop(); cout << "\ns1:" << s1 << "\n";
```

```
cout << "\ns1.pop(): " << s1.pop(); cout << "\ns1:" << s1 << "\n";
```

```
// cout << "\ns1.pop(): " << s1.pop(); cout << "\ns1:" << s1 << "\n";
```

```
cout << "s1 = reverse(s2);\n";
```

```
s1 = reverse(s2);
```

// тук копиращият конструктор се вика 2 пъти

// 1-во защото подаваме стек, като параметър

// 2-ро защото функцията връща стек

// накрая се вика операторът за присвояване

```
cout << "s1:" << s1 << endl;
```

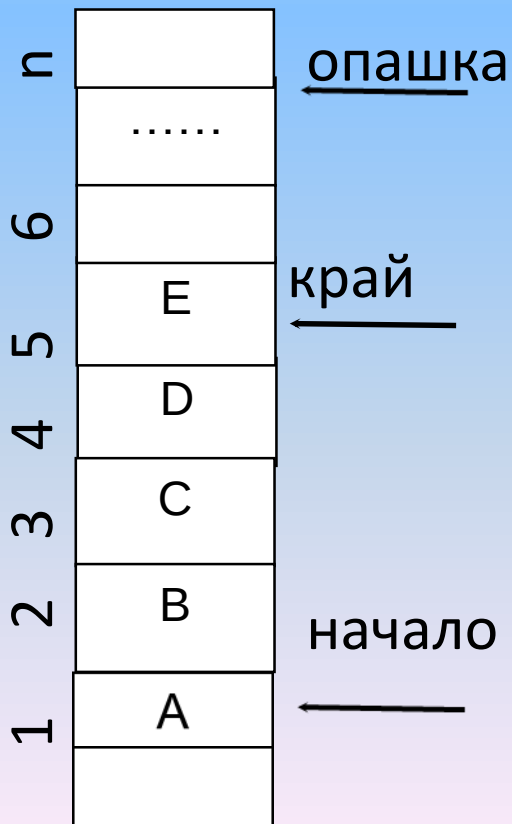
```
cout << "s2:" << s2 << endl;
```

```
return 0;
```

```
}
```


Структури от данни. Опашка (Queue)

- **Опашка** - линейна структура в която операциите се извършват и в двата края – единият се нарича **край**, а другият **начало**. В края се записва, а в началото се чете. Опашката работи по принципа **FIFO - First In First Out**.



Опашката има широко приложение. Пример: в един *mail* сървър има опашка, в която се пазят писмата, които трябва да бъдат изпратени. Писмото влязло първо в опашката се изпраща най-напред.

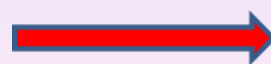


Структури от данни. Опашка (Queue)

Пример: демонстриращ използването на клас Queue:

```
#include <iostream>
#include <fstream>
#include "queue.h"
using namespace std;
template <class T>
Queue<T> increase(Queue<T> a) {
    T x;
    Queue<T> b;
    while(a.pop(x)) b.push(x + 1);
    return b;
}
int main(){
    Queue<int> q1;
    q1.push(2); cout << "push 2\n";
    q1.push(4); cout << "push 4\n";
    q1.push(6); cout << "push 6\n";
    cout << "q1:" << q1;
    cout << endl;
```

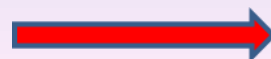
// тук се вика конструкторът по подразбиране





Структури от данни. Опашка (Queue)

```
cout << "Queue<int> q2(q1), q3;\nq3 = q1;\nq3.push(7);\n";
Queue<int> q2(q1), q3;                                // тук се вика копиращият конструктор
q3 = q1;                                              // тук се вика операторът за присвояване
q3.push(7);
cout << "q1:" << q1 << endl;                        // тук се вика операторът за извеждане
cout << "q2:" << q2 << endl;
cout << "q3:" << q3 << endl;
int a;
q1.pop(a);
cout << "\nq1.pop(a), a: " << a;
cout << "\nq3.writeFile\n";
q3.writeToFile("test.mp3");
cout << "q1.readFile\n";
q1.readFile("test.mp3");
cout << "q1:" << q1 << endl;
cout << "q2 = q1 = q3\n"; q2 = q1 = q3;
cout << "\nq1:" << q1;
cout << "\nq2:" << q2;
cout << "\nq3:" << q3;
```





Структури от данни. Опашка (Queue)

```
cout << "\nq2.push(15)\n"; q2.push(15);
```

```
cout << "\nq1:" << q1;
```

```
cout << "\nq2:" << q2;
```

```
cout << "\nq3:" << q3;
```

```
cout << "\nq1.pop(): " << q1.pop(); cout << "\nq1:" << q1 << "\n";
```

```
cout << "\nq1.pop(): " << q1.pop(); cout << "\nq1:" << q1 << "\n";
```

```
cout << "\nq1.pop(): " << q1.pop(); cout << "\nq1:" << q1 << "\n";
```

```
cout << "\nq1.pop(): " << q1.pop(); cout << "\nq1:" << q1 << "\n";
```

```
// cout << "\nq1.pop(): " << q1.pop(); cout << "\nq1:" << q1 << "\n";
```

```
cout << "q1 = increase(q2);\n";
```

```
q1 = increase(q2);
```

// тук копиращият конструктор се вика 2 пъти

// 1-во защото го подаваме, като параметър

// 2-ро защото функцията връща опашка

// накрая се вика операторът за присвояване

```
cout << "q1:" << q1 << endl;
```

```
cout << "q2:" << q2 << endl;
```

```
return 0;
```

```
}
```



Структури от данни. Опашка (Queue)

Освен стандартната опашка има още **опашка с приоритети**.

При нея изваждането на елементите става в някакъв ред, в зависимост от приоритета им.

Това може да се реализира, като поддържаме опашката сортирана - при всяко вмъкване търсим точното място на елемента и го поставяме там.

Друг вариант е елементите да са подредени по реда си на влизане, а при изваждането да търсим този с най-голям приоритет и да извадим него.



Структури от данни. Дек (Deque)

Декът представлява **опашка**, в която могат да се добавят или премахват елементи **от двата края**.

Има два вида дек:

- **с ограничен вход**, при който елементите могат да се премахват и от двата края, но вмъкването им става само от единия;
- **с ограничен изход**, при който вмъкване може да се прави и в двата края, но премахването става само от единия.



Структури от данни. Списък (List)

- **Списък** - линейна структура при която записът и четенето могат да се правят на произволно място. Достъпът до елементите е **последователен**. В паметта елементите на списъка се помещават в клетки с **произволни адреси** (*а не в последователни, както е напр. при масив*).

Видове списъци:

- **Еднопосочен** (едносвързан) – с едно указателно поле за следващ елемент;
- **Двупосочен** (двусвързан) – с две указателни полета (за предишен и за следващ елемент).

И двата вида имат **цикличен** (указателят сочи 1-ят елемент) и **нецикличен** (указателят на последния елемент е 0 или nil) вариант



Структури от данни. Списък (List)

Нецикличният вариант може да бъде с или без заглавен елемент.

Заглавният елемент съдържа **името** на списъка и указателят му сочи 1-ят елемент на списъка.

Всеки списък може да бъде в два варианта:

- Със сортирани елементи;
- С несортирани елементи по определен ключ.

Свързаният списък е структура от данни изградена от елементи, всеки от които съдържа някакви **данни + информация**, къде се намира следващият елемент от списъка.

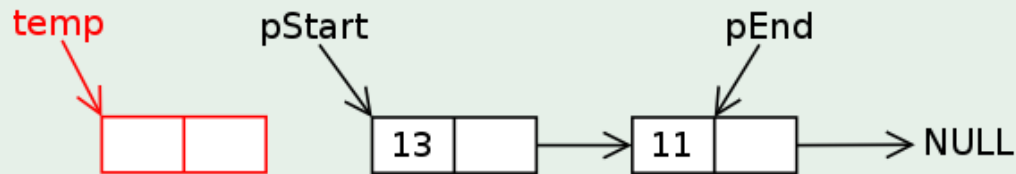


Структури от данни. Списък (List)

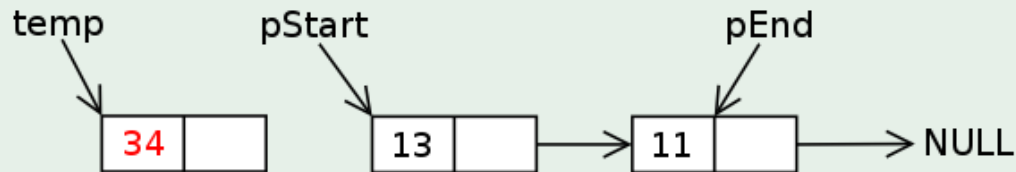
Вмъкване на елемент в началото на списъка:

```
insertFirst(T aData)  aData = 34
```

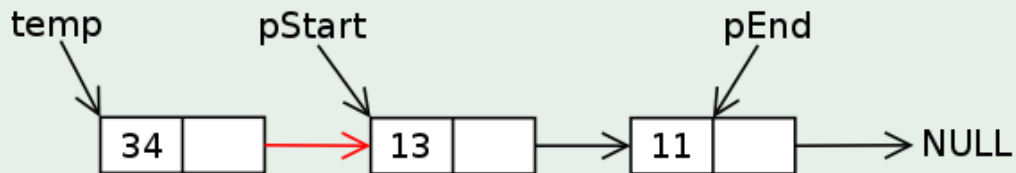
```
Elem<T>*temp = new Elem<T>;
```



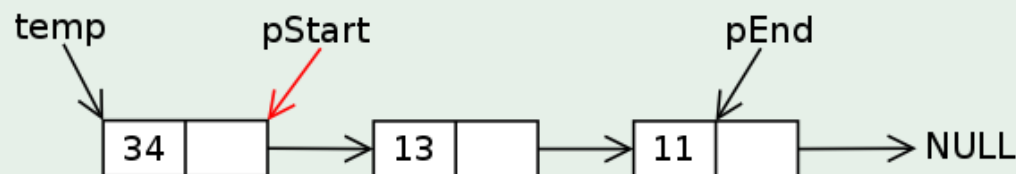
```
temp->data = aData;
```



```
temp->next = pStart;
```



```
pStart = temp;
```



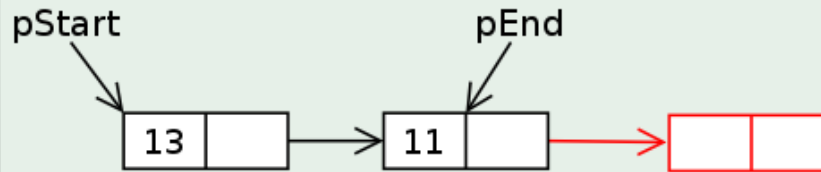


Структури от данни. Списък (List)

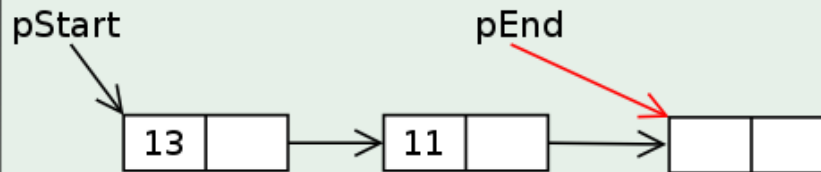
Вмъкване на елемент в края на списъка:

```
insertLast(T aData)  aData = 34
```

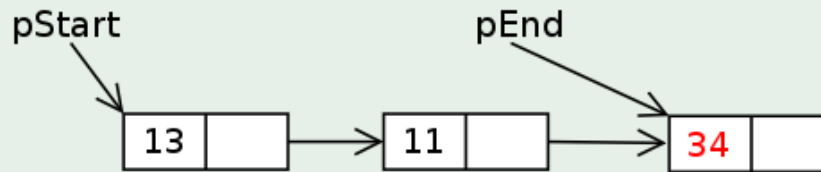
```
pEnd->next = new Elem<T>;
```



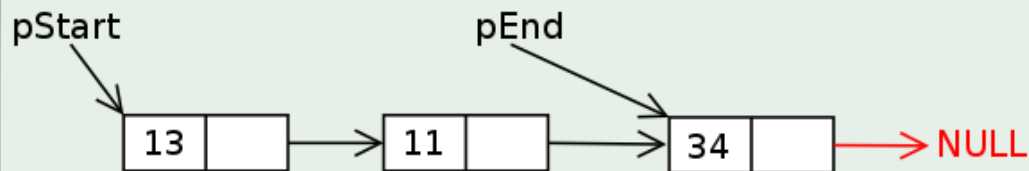
```
pEnd = pEnd->next;
```



```
pEnd->data = aData;
```



```
pEnd->next = NULL;
```

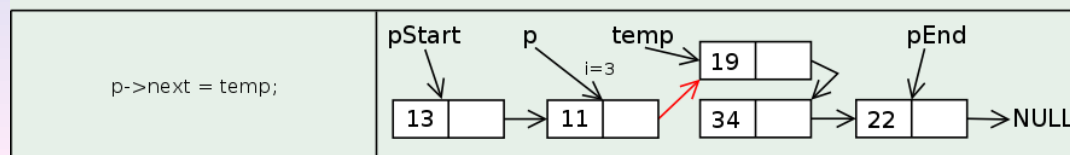
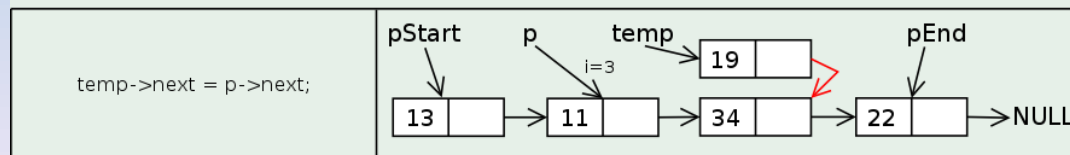
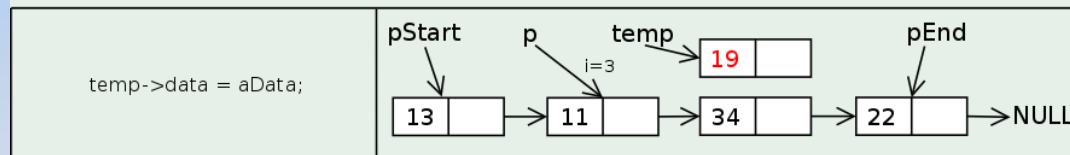
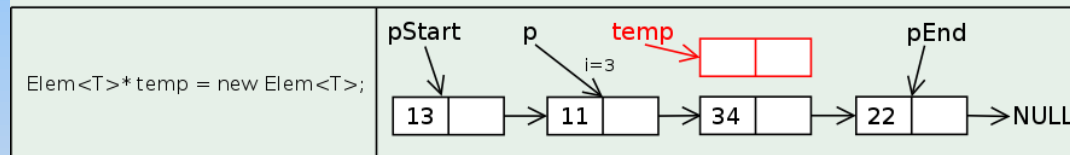
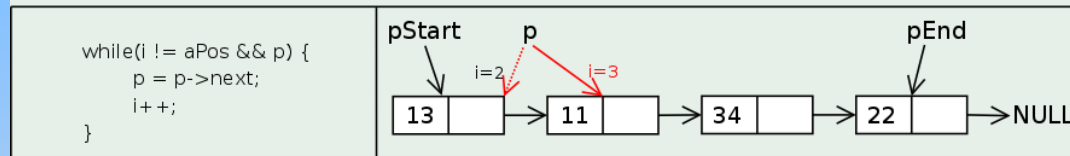
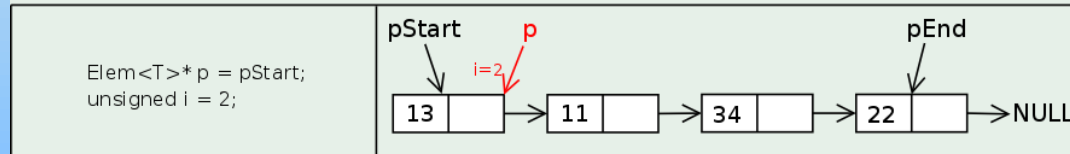




Структури от данни. Списък (List)

Вмъкване на елемент на зададена позиция. Елементите от тази позиция (включително) нататък се преместват към края на списъка

`insertAt(unsigned aPos, T aData)    aPos=3,    aData = 19`



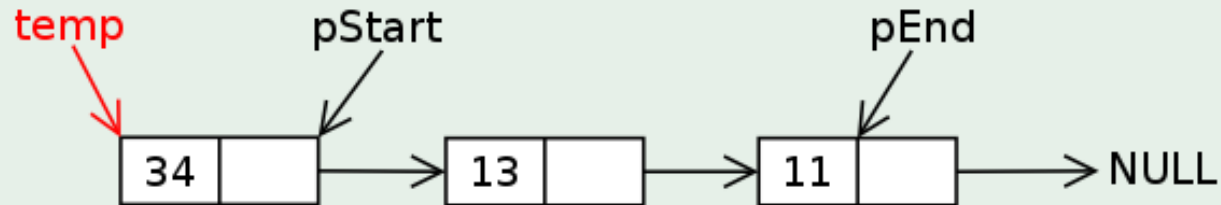


Структури от данни. Списък (List)

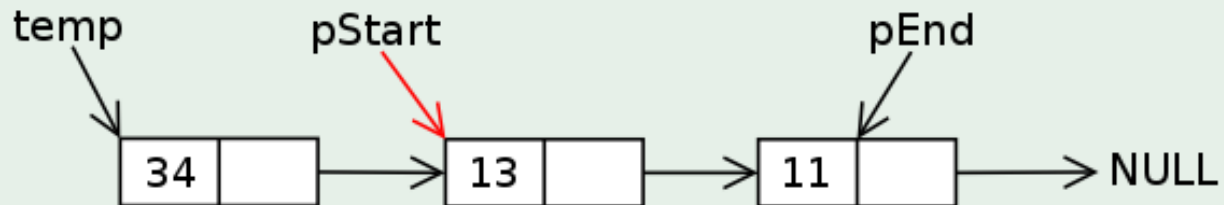
Премахване на елемент в началото на списъка:

`removeFirst()`

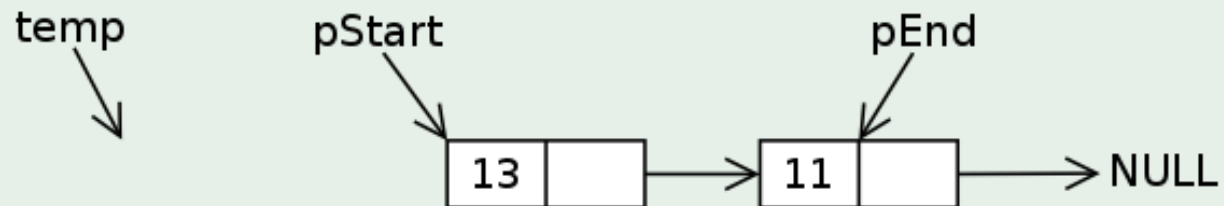
`Elem<T>*temp = pStart;`



`pStart = pStart->next;`



`delete temp;`

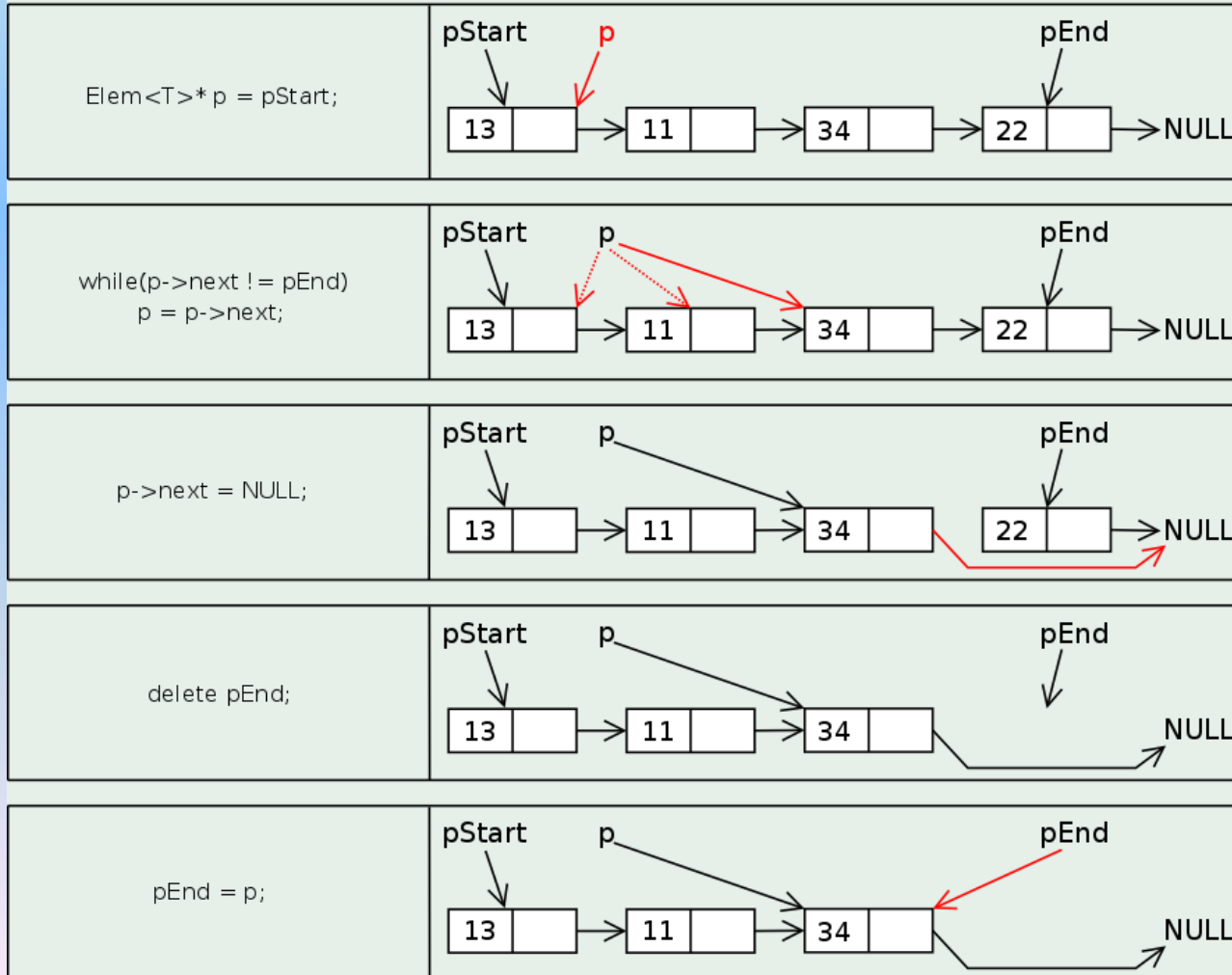




Структури от данни. Списък (List)

Премахване на елемент в края на списъка:

`removeLast()`



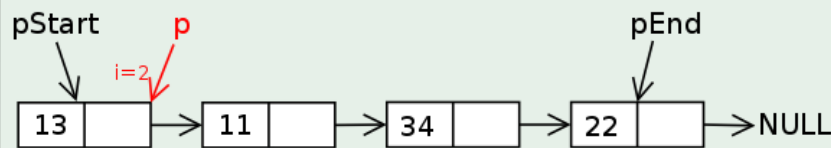


Структури от данни. Списък (List)

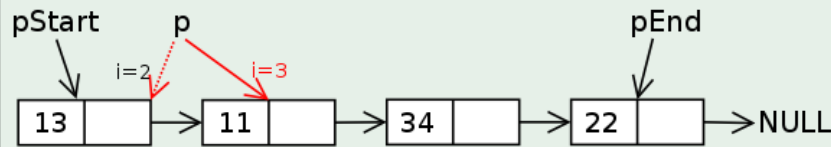
Премахване на елемент на зададена позиция:

removeAt(unsigned aPos) aPos = 3

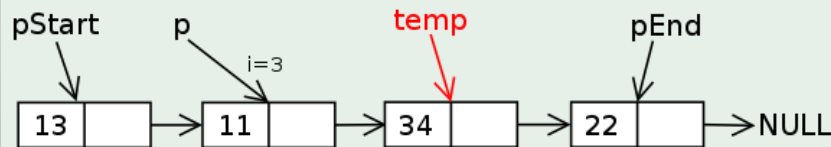
```
Elem<T>* p = pStart;  
unsigned i = 2;
```



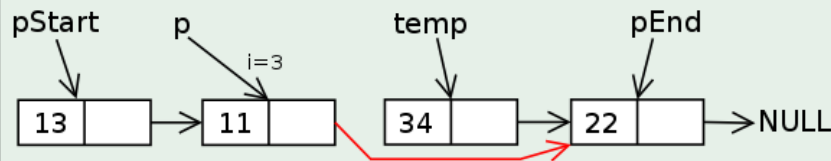
```
while(i != aPos && p->next) {  
    p = p->next;  
    i++;  
}
```



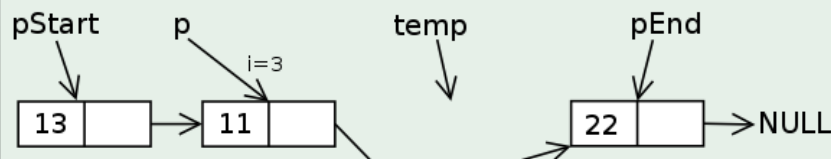
```
Elem<T>* temp = p->next;
```



```
p->next = p->next->next;
```



```
delete temp;
```

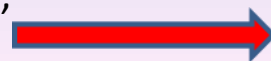




Структури от данни. Списък (List)

Пример: демонстриращ използването на клас List:

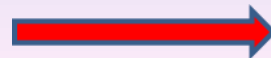
```
#include <iostream>
#include <fstream>
#include "list.h"          // кавичките казват на препроцесора най-напред да търси файла
list.h                   // в директорията, където е текущият файл
using namespace std;
int main() {
    List<int> l1;          // тук се вика конструкторът по подразбиране
    cout << "is empty: " << l1.isEmpty() << endl;
    cout << "insertFirst 11\n";
    l1.insertFirst(11);
    cout << "list: ";
    l1.print();
    cout << "\n";
    cout << "insertLast 33\n";
    l1.insertLast(33);
    cout << "list: " << l1 << "\n";    // тук се вика операторът за извеждане
    cout << "is empty: " << l1.isEmpty() << "\n\n";
```





Структури от данни. Списък (List)

```
cout << "l2(l1)\n";  
List<int> l2(l1); // тук се вика копиращият конструктор  
cout << "l2.insertAt(2, 22)\n";  
l2.insertAt(2, 22);  
cout << "l1[1] = 0\n";  
l1[1] = 0; // тук се вика операторът за достъп  
cout << "l2: " << l2 << "\n";  
cout << "l1: " << l1 << "\n\n";  
List<int> l3;  
cout << "l3 = l2\n";  
l3 = l2; // тук се вика операторът за присвояване  
cout << "l3.removeFirst()\n";  
l3.removeFirst();  
cout << "l3.removeAt(2)\n";  
l3.removeAt(2);  
cout << "l3: " << l3 << "\n";  
cout << "l2: " << l2 << "\n";  
cout << "l1: " << l1 << "\n\n";
```





Структури от данни. Списък (List)

```
cout << "l2.writeToFile(\"list\");" << endl;
```

```
l2.writeToFile("list");
```

```
cout << "l1.removeAll\n";
```

```
l1.removeAll();
```

```
cout << "l1: " << l1 << "\n";
```

```
cout << "l1.isEmpty(): " << l1.isEmpty() << endl;
```

```
cout << "l1.readFile(\"list\")\n\n";
```

```
l1.readFile("list");
```

// тук използваме итератора, за да отпечатаме списъка

```
Elem<int>* p;
```

```
l1.iteratorReset();
```

```
while(p = l1.getNext()) cout << p->data << ' ';
```

```
cout<< "\n";
```





Структури от данни. Списък (List)

// ако вместо да използваме итератора напишем

```
for(int i = 1; i <= 3; i++) cout << l1[i] << ' ';
```

```
cout<< "\n";
```

// обхождането на списъка ще става многократно по-бавно,

// особено при големи списъци, защото при всяко обръщение към оператора []

// се тръгва от началото на списъка

```
ofstream f("a");
```

```
f << l1 << " | " << l2 << " | " << l3;
```

// тук отново се вика операторът за извеждане

```
f.close();
```

// но потокът е файлов

```
return 0;
```

// след като свърши областта на видимост, в която са декларирани обектите,

// се викат деструкторите им и се унищожават

}